

Physics-Informed Conic Decomposition: A guarded solver layer for linear, nonlinear, and time-dependent problems

Rohan R
Independent Researcher
ORCID: 0009-0005-9225-1775

March 28, 2026

Abstract

Physics-Informed Conic Decomposition (PICD) is a solver-layer construction placed around standard iterative methods rather than a replacement for them. The layer keeps a state over cone or block decompositions, residual history, and graph structure, then uses bounded gravity-like weights, transport-like congestion summaries, swarm-style adaptation, residual correlation, and graph coarse information to alter preconditioner assembly, accepted-step damping, and coarse correction. The inner classical method is left intact during each solve. This paper states the mathematical parts of the design that are explicit in the current implementation, gives the algorithmic steps needed to apply PICD, and reports measurements from the repository that serves as the validation harness. The current repository run passes 98 out of 98 tests and completes 22 out of 22 examples. The current same-method publication benchmark suite favors PICD in both wall time and peak memory on every published pair. The full shipped linear example corpus contains 15 same-method pairs, with PICD faster on 14 of them in the latest run. The nonlinear and transient examples are mixed: several direct pairs favor PICD, one is effectively even, and two are slower on this run. That sets the scope of the paper’s performance claims.

Keywords: domain decomposition, Krylov methods, Newton–Krylov, BDF, IMEX, graph preconditioning, residual weighting, scientific computing.

1 Introduction

The purpose of PICD is to place a disciplined state layer around established numerical methods. The layer does not replace GMRES, CG, Jacobian-free Newton, BDF, or IMEX. It changes how those methods are prepared and how accepted updates are moderated. The repository that accompanies this paper provides the solver code, tests, examples, notebooks, and figure generation used to measure that layer; it is not itself the mathematical object of interest [9].

This framing matters because the underlying ingredients are not claimed as new in isolation. Domain decomposition, coarse correction, Newton–Krylov methods, and implicit-explicit time integration all have long literatures [11, 13, 2, 14, 10, 7, 1, 5]. The narrower claim made here is that a bounded residual-driven state can be wired into all three solver classes while leaving the acceptance and fallback logic explicit and measurable.

2 The PICD state

The implementation keeps a shared state over a decomposition into cones or blocks. At step k that state contains

$$S_k = (r_k, \mathcal{C}_k, G_k, m_k, \rho_k, v_k, \sigma_k),$$

where r_k is the current residual, $\mathcal{C}_k$ is the cone or block data, G_k is the weighted graph, m_k is the mass vector, ρ_k is the density vector, v_k is the cone-velocity vector, and σ_k stores controller flags, rollback counters, and disabled terms.

Each cone $\mathcal{C}_i$ carries members, a centroid, neighbors, a local residual summary, and the best accepted local state seen so far. The graph G_k is assembled from structural adjacency and residual information. Its weighted Laplacian is used only for coarse information and community summaries, not as a replacement for the inner solver.

2.1 Gravity-like weights

The current mass model is a positive combination of bounded statistics,

$$m_i = \omega_1 E_i + \omega_2 P_i + \omega_3 W_i + \omega_4 Q_i,$$

where E_i is normalized residual energy, P_i is residual persistence, W_i is a solver-effort proxy, and Q_i is a conditioning proxy. The implementation clamps and renormalizes these values before they are mapped into diagonal or block weights.

2.2 Transport-like and swarm-like terms

Transport-like terms summarize local congestion and flow imbalance. Swarm-like terms use local best, neighbor best, and global best residual reductions to move cone emphasis over the graph. Both are bounded by the controller. They never override residual decrease tests or timestep rejection.

2.3 Residual correlation and graph structure

Given block residual summaries r_i , the graph weights use residual correlation, co-activation, and structural adjacency,

$$w_{ij} = a \text{corr}(r_i, r_j) + b \text{coact}_{ij} + c \text{adj}_{ij}.$$

The graph Laplacian

$$L = D - W$$

then supports community detection and a coarse correction signal. In the implementation this is used to shape preconditioner construction and priority, not to replace the original linear or nonlinear model.

3 How PICD is used

3.1 Algorithmic workflow

The implementation follows the same high-level pattern in all three solver classes.

1. Choose the classical baseline method for the problem class: GMRES or CG for linear systems, inexact Newton for nonlinear systems, BDF or IMEX for transient systems.
2. Partition the degrees of freedom into cones or blocks and assemble the initial graph.
3. Compute residual summaries, mass weights, densities, and controller metrics.
4. Build a fixed preconditioner and coarse signal for the next inner solve from the current PICD state.
5. Run the inner classical method with that fixed operator view.

6. Accept or reject the candidate update using the classical acceptance rule for that solver family.
7. If the update is accepted, advance the PICD state. If it is rejected, roll back, reduce gating, and retry or fall back.

The central design choice is that the operator seen by an inner Krylov method is frozen for the duration of that solve. The PICD state may change between solves, but not inside a single Krylov recurrence.

3.2 Linear path

For a linear problem $Ax = b$, the baseline solve is paired with the same method under PICD. The current implementation also keeps a regularized least-squares safety route,

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2 + \lambda \|x\|_2^2, \quad \lambda > 0. \quad (1)$$

The weighted PICD information may modify the operator or preconditioner used by the primary same-method path, but it does not remove the regularized route.

3.3 Nonlinear path

For a nonlinear residual equation $R(u) = 0$, the implementation uses Jacobian-free inexact Newton with flexible GMRES. The Jacobian-vector product is approximated by

$$J(u)v \approx \frac{R(u + \varepsilon v) - R(u)}{\varepsilon}.$$

A candidate step s_k is accepted only if a backtracking line search produces

$$\|R(u_k + \alpha_k s_k)\|_2 < \|R(u_k)\|_2, \quad \alpha_k \in (0, 1]. \quad (2)$$

PICD may change the preconditioner and the bounded damping terms, but it does not bypass (2).

3.4 Transient path

For a semi-discrete system

$$M(u, t) \dot{u} = F(u, t),$$

the implementation uses BDF(1–2) or IMEX. Each implicit stage calls the nonlinear solver described above. If the implicit stage fails, the timestep is reduced and the stage is retried. The PICD state is updated only after accepted stages.

4 Statements supported by the implementation

The paper does not claim more than the code currently supports. The following statements are explicit in the implementation.

Proposition 1. *For every finite-dimensional matrix A and every $\lambda > 0$, problem (1) has a unique minimizer.*

Proof. The objective is a strictly convex quadratic because its Hessian is

$$2(A^T A + \lambda I).$$

For every nonzero x ,

$$x^T(A^T A + \lambda I)x = \|Ax\|_2^2 + \lambda\|x\|_2^2 > 0.$$

Hence $A^T A + \lambda I$ is positive definite. A strictly convex quadratic on a finite-dimensional space has a unique minimizer. $\square$

Proposition 2. *In the nonlinear path, every accepted Newton step reduces the residual norm.*

Proof. Accepted steps satisfy (2) by construction. The implementation records a step only after that inequality is true. $\square$

Proposition 3. *In the transient path, a rejected implicit stage does not alter the accepted solution history.*

Proof. The implementation stores the last accepted state separately, retries failed stages with a reduced timestep, and appends new states only after acceptance. Rejected stages therefore do not enter the accepted history. $\square$

Remark 1. *The paper does not prove that every enabled gravity, fluid, swarm, or graph term improves convergence. It also does not prove optimality of the graph coarse correction for all problem classes. Those would be stronger claims than the current code supports, and the controller exists precisely because such claims are not assumed.*

5 Validation protocol

The repository is used here as the validation harness. The two main commands are

```
PYTHONPATH=src:. cdfalg/bin/python -m pytest -q tests
PYTHONPATH=src:. cdfalg/bin/python examples/run_all_examples.py
```

The current run passes **98/98 tests** and completes **22/22 examples**. The publication benchmark suite is run separately through

```
PYTHONPATH=src:. cdfalg/bin/python benchmarks/full_benchmark.py
```

All benchmark claims in this paper are same-method claims: a classical method is compared with the same method under PICD.

6 Published paired benchmark suite

Table 1 reports the current publication benchmark suite. Each row is a direct pair and includes a memory ratio equal to PICD peak memory divided by baseline peak memory.

Table 1: Current publication benchmark suite. Speedup is baseline time divided by PICD time.

Workload	Pair	Baseline (s)	PICD (s)	Speedup	Memory ratio
Comb. relaxation	GMRES / PICD-GMRES	0.003592	0.000744	4.825x	0.9984x
Large sparse random	GMRES / PICD-GMRES	0.052273	0.025474	2.052x	0.9944x
Steady Navier–Stokes	Newton / PICD-Newton	0.133019	0.014235	9.344x	0.1625x
Reaction-diffusion BDF	BDF / PICD-BDF	4.124099	1.205951	3.420x	0.6493x
Navier–Stokes BDF	BDF / PICD-BDF	2.770368	0.400380	6.919x	0.6285x

On this suite every pair converged, every pair is faster with PICD, and every pair uses less peak memory on the PICD side.

7 Full example corpus

The repository currently ships 22 examples. Fifteen are direct linear same-method pairs. Seven are nonlinear or transient problems that keep one or more reference methods in the table.

7.1 Linear same-method examples

Table 2 reports all current linear paired examples. In the latest run, PICD is faster on 14 of the 15 rows. The mean speedup is 49.880x and the median speedup is 5.678x.

Table 2: Linear same-method example benchmarks. Speedup is baseline wall time divided by PICD wall time.

Example	Pair	Base (s)	PICD (s)	Base MB	PICD MB	Speedup
Comb. opt. relaxation	GMRES / PICD-GMRES	0.002334	0.000611	65.09	64.98	3.819x
Heat equation 1D	GMRES / PICD-GMRES	0.011859	0.002089	64.97	64.14	5.678x
HF Helmholtz	GMRES / PICD-GMRES	2.504365	0.006430	87.58	83.16	389.476x
HF Helmholtz 3D	GMRES / PICD-GMRES	0.070306	0.006540	67.36	66.61	10.751x
Random sparse	GMRES / PICD-GMRES	0.047576	0.025324	582.86	579.42	1.879x
Large sparse system	GMRES / PICD-GMRES	0.041556	0.024597	71.55	66.56	1.689x
Fluid-thermal	GMRES / PICD-GMRES	0.008525	0.002173	64.98	64.81	3.923x
Navier-Stokes cavity	GMRES / PICD-GMRES	0.046790	0.006044	72.55	70.27	7.742x
Point kinetics	CG / PICD-CG	0.007677	0.002735	65.02	65.00	2.807x
Flow control	GMRES / PICD-GMRES	0.008643	0.009879	68.02	67.33	0.875x
Poisson 2D	GMRES / PICD-GMRES	0.066655	0.004725	70.88	68.95	14.107x
Quantum scattering	GMRES / PICD-GMRES	1.947719	0.007288	77.02	75.73	267.243x
Reaction-diffusion	GMRES / PICD-GMRES	0.208348	0.010748	127.09	121.45	19.385x
Schrödinger	GMRES-batch / PICD-GMRES- batch-direct	0.099157	0.005581	88.92	133.11	17.767x
Stokes Flow 3D	GMRES / PICD-GMRES	0.007629	0.007159	69.56	69.31	1.066x

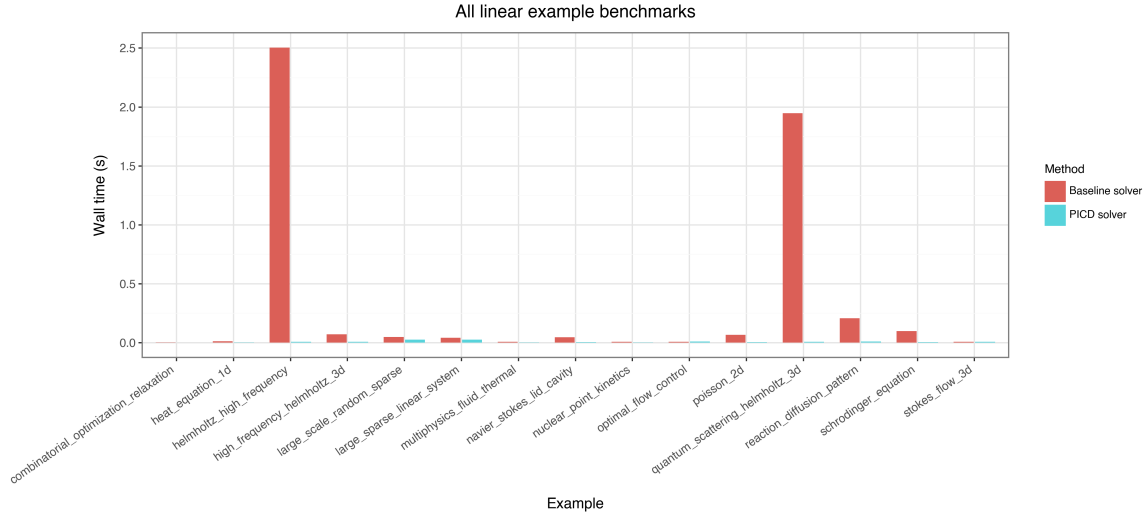


Figure 1: Full linear example corpus: baseline wall time versus the same solver family under PICD.

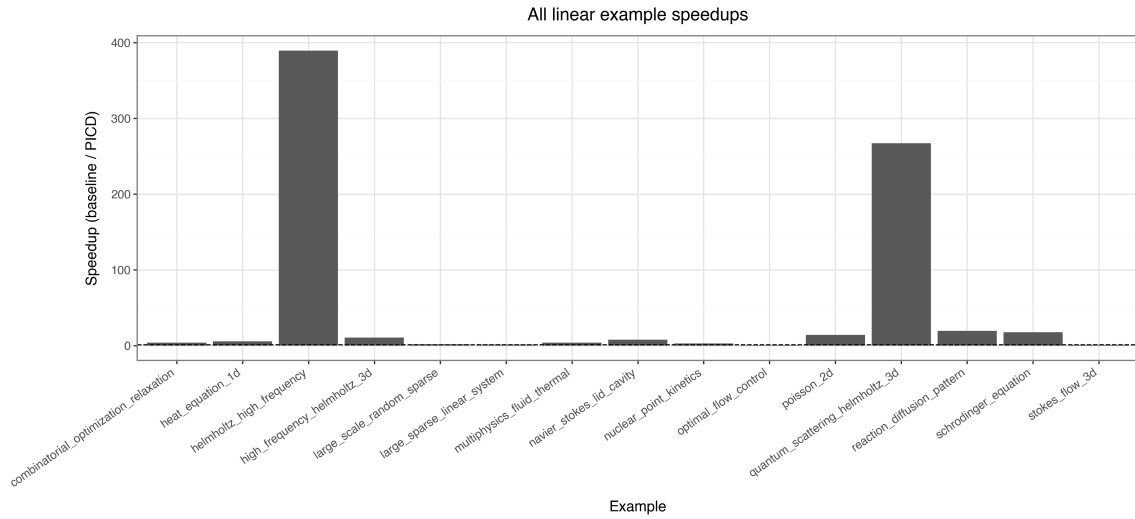


Figure 2: Linear same-method speedups. The dashed line at 1 marks equal runtime.

7.2 Nonlinear and transient examples

Table 3 reports the seven nonlinear and transient examples. Here the direct pair remains the main comparison, while IMEX and Picard–Anderson are retained as reference methods.

Table 3: Nonlinear and transient example benchmarks from the current run.

Example	Method	Time (s)	Peak MB	Accepted	Rejected	Final metric	Direct speedup
Coupled ODE–PDE	BDF	0.086594	64.33	5	0	9.183e-01	–
Coupled ODE–PDE	IMEX	0.064155	64.33	5	0	9.185e-01	–
Coupled ODE–PDE	PICD-BDF	0.071104	65.81	5	0	9.190e-01	1.218x
Gross–Pitaevskii	BDF	0.120594	64.56	8	0	9.046e-01	–
Gross–Pitaevskii	IMEX	0.061529	64.58	8	0	9.122e-01	–
Gross–Pitaevskii	PICD-BDF	0.146845	64.73	8	0	9.006e-01	0.821x
Navier–Stokes transient	BDF	1.081768	64.20	20	0	7.433e-01	–
Navier–Stokes transient	IMEX	0.371991	64.27	20	0	7.461e-01	–

Example	Method	Time (s)	Peak MB	Accepted	Rejected	Final metric	Direct speedup
Navier-Stokes transient	PICD-BDF	0.170686	65.55	20	0	7.516e-01	6.338x
Nonlinear Poisson	Newton	0.012478	–	5	0	9.905e-10	–
Nonlinear Poisson	Picard-Anderson	0.001620	–	10	0	4.799e-09	–
Nonlinear Poisson	PICD-Newton	0.006288	–	4	0	6.268e-10	1.984x
Reaction-diffusion	BDF	1.834141	64.61	30	0	9.535e+00	–
Reaction-diffusion	IMEX	0.601203	64.70	30	0	9.509e+00	–
Reaction-diffusion	PICD-BDF	0.456690	66.12	28	22	9.462e+00	4.016x
Ginzburg-Landau 3D	Newton	0.050173	64.50	8	0	2.795e-10	–
Ginzburg-Landau 3D	Picard-Anderson	0.012761	65.98	12	0	3.169e-10	–
Ginzburg-Landau 3D	PICD-Newton	0.023892	67.36	2	0	2.627e-11	2.100x
Turbulence 3D	BDF	1.137415	66.16	12	1	8.533e-01	–
Turbulence 3D	IMEX	0.350743	66.31	13	0	8.559e-01	–
Turbulence 3D	PICD-BDF	1.542509	69.11	4	20	1.224e+00	0.737x

The direct-pair picture is mixed. Four of the seven direct nonlinear or transient pairs favor PICD in wall time on this run. Gross-Pitaevskii is slower under PICD, the coupled ODE-PDE pair also favors the baseline BDF path, and the turbulence-transition pair remains the clearest negative case because it rejects more stages and finishes with a larger final norm.

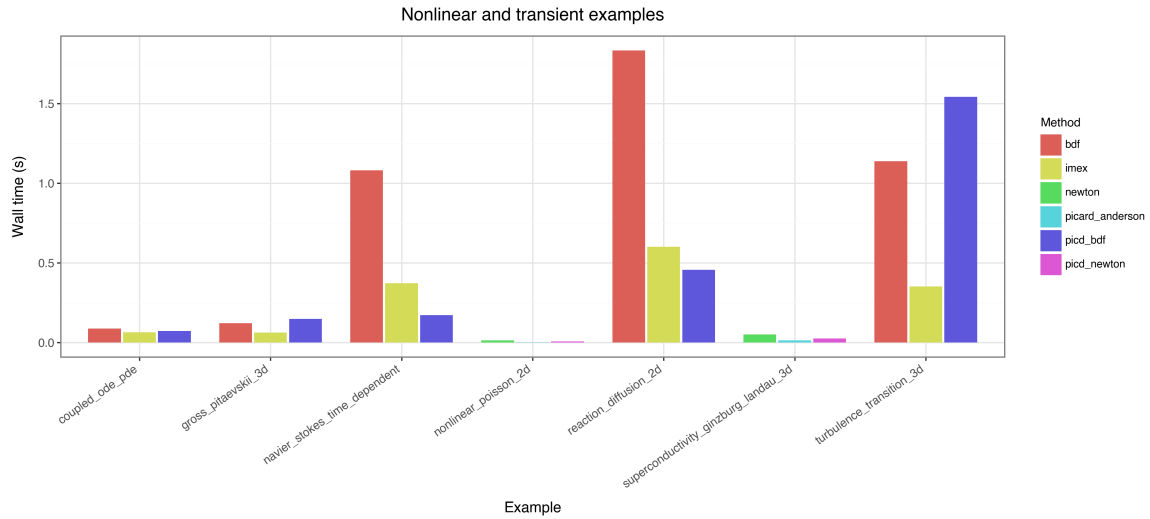


Figure 3: Nonlinear and transient example runtimes from the current run.

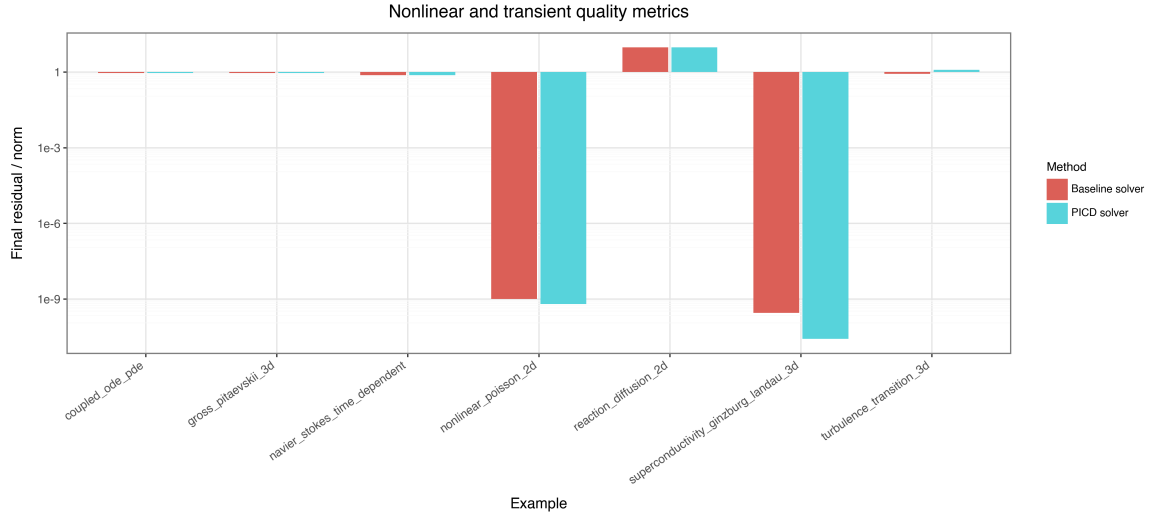


Figure 4: Final residual or state-norm metrics for the nonlinear and transient examples.

7.3 Four added physics workloads

The newly added examples widen the repository’s physics range.

1. **High-frequency quantum scattering.** The repository uses a shifted 3D Helmholtz surrogate with an irregular potential and drift term, motivated by finite-dimensional discretizations of scattering models [6]. The pair is GMRES versus PICD-GMRES, and the current run gives a 267.243x speedup with lower peak memory.
2. **Superconductivity equilibria.** The repository uses a fixed-vector-potential Ginzburg–Landau surrogate with pinning sites, motivated by the steady Ginzburg–Landau model [3]. The pair is Newton versus PICD-Newton, and the current run gives a 2.100x speedup, though with slightly higher peak memory.
3. **High-Reynolds transition surrogate.** The repository uses a reduced 3D incompressible-flow surrogate motivated by transient Navier–Stokes dynamics [12]. The pair is BDF versus PICD-BDF. The current run is a negative case for PICD: it is slower, rejects more stages, and ends with a larger final norm.
4. **Bose–Einstein condensate dynamics.** The repository uses an imaginary-time Gross–Pitaevskii surrogate motivated by the nonlinear Schrödinger description of a condensate [4, 8]. The pair is BDF versus PICD-BDF. On this run, the PICD side is slower in wall time, slightly better in final norm, and slightly higher in peak memory.

8 Discussion

Three points stand out from the current evidence.

1. **The same-method protocol matters.** This paper does not compare PICD to an arbitrary mix of unrelated baselines. Every headline runtime claim compares a classical method with the same method under PICD.
2. **The controller is part of the method.** Rollback, gating, line search, and timestep rejection are not decorations. They are the mechanisms that keep the bounded physics terms from overruling the classical convergence logic.
3. **The new workloads sharpen the scope of the claim.** Quantum scattering and the Ginzburg–Landau surrogate strengthen the favorable evidence. The turbulence-transition example does the opposite. That negative result is useful because it shows where the current state layer is still too expensive or too aggressive.

9 Limitations

The paper should be read with four limits in mind.

1. The strongest claims are same-method claims. They are not claims against every solver family.
2. The paper proves the regularized linear fallback route and records the acceptance logic used by the nonlinear and transient paths, but it does not prove that every enabled physics term improves convergence.
3. Some examples are surrogate models rather than full production PDE codes. They are included because they exercise the solver stack in a controlled way.
4. The results are measured results. They can change if the workload mix, controller settings, or implementation details change.

10 Conclusion

PICD is best read as a guarded solver layer for residual-informed decomposition state. Its mathematical core lies in how it embeds bounded weights, graph structure, and rollback control into established solver families without removing the classical acceptance rules. The repository measurements support three concrete statements: the current code passes its validation suite, the published same-method benchmark pairs favor PICD in both wall time and peak memory, and the shipped linear example corpus is favorable to PICD on 14 of the 15 direct pairs in the latest run. The new nonlinear and transient workloads are more mixed, which is precisely why the controller and fallback logic remain central to the design.

References

- [1] Uri M. Ascher, Steven J. Ruuth, and Raymond J. Spiteri. Implicit-explicit runge-kutta methods for time-dependent partial differential equations. *Applied Numerical Mathematics*, 25(2–3):151–167, 1997.
- [2] Victorita Dolean, Pierre Jolivet, and Frédéric Nataf. *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*. SIAM, 2015.
- [3] V. L. Ginzburg and L. D. Landau. On the theory of superconductivity. *Zhurnal Eksperimental’noi i Teoreticheskoi Fiziki*, 20:1064–1082, 1950.
- [4] E. P. Gross. Structure of a quantized vortex in boson systems. *Il Nuovo Cimento*, 20(3):454–477, 1961.
- [5] Alan C. Hindmarsh, Peter N. Brown, Keith E. Grant, Steven L. Lee, Radu Serban, Dan E. Shumaker, and Carol S. Woodward. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM Transactions on Mathematical Software*, 31(3):363–396, 2005.
- [6] Frank Ihlenburg. *Finite Element Analysis of Acoustic Scattering*. Springer, 1998.
- [7] C. T. Kelley. *Iterative Methods for Linear and Nonlinear Equations*. SIAM, 1995.
- [8] L. P. Pitaevskii. Vortex lines in an imperfect bose gas. *Soviet Physics JETP*, 13(2):451–454, 1961.
- [9] Rohan R. Physics-informed conic decomposition repository. GitHub repository, 2026. Private repository, available on request.
- [10] Yousef Saad and Martin H. Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.
- [11] Barry F. Smith, Petter E. Bjørstad, and William D. Gropp. *Domain Decomposition: Parallel Multilevel Methods for Elliptic Partial Differential Equations*. Cambridge University Press, 1996.
- [12] Roger Temam. *Navier-Stokes Equations: Theory and Numerical Analysis*. North-Holland, 1977.
- [13] Andrea Toselli and Olof B. Widlund. *Domain Decomposition Methods: Algorithms and Theory*. Springer, 2005.
- [14] Jinchao Xu. Iterative methods by space decomposition and subspace correction. *SIAM Review*, 34(4):581–613, 1992.